

C RTP は「もう古い」の？

モダン C++ での立ち位置

— コンセプト / deducing this と C RTP —

ゲーム開発のための C++ 講義 (第9章の補足)

この補講の問い

第9章で **deducing this** **C++23** を、
別の回で **コンセプト** **C++20** を学んだ。すると、こんな疑問が浮かぶ。

🤔 「コンセプトを覚えたら、**C RTP** はもう時代遅れなのでは？」

いい疑問！ でも答えは少しだけ意外。結論を先に言うと——

- C RTP を置き換えるのは、実は **コンセプトではない**
- 置き換えるのは、**第9章で習った deducing this** のほう

なぜそうなるのか、用語をゆっくりほどきながら見ていこう。

復習：C RTP ってどんな技術？

Curiously **R**ecurring **T**emplate **P**attern（不思議な再帰テンプレート）。
「基底クラスに、**自分（派生）の型を教えて**継承する」独特な書き方。

```
template <class Derived>
struct Shape {
    void draw() const {
        // 自分を派生型に“キャストし直して”呼ぶ
        static_cast<const Derived*>(this)→draw_impl();
    }
};

struct Circle : Shape<Circle> { // ← 自分自身 Circle を渡して継承（ここが名前の由来）
    void draw_impl() const { /* 円を描く */ }
};
```

- `virtual` を使わずに「派生ごとの処理」を呼び分ける（= **静的多態**：速い）
- でも `Shape<Circle>` や `static_cast` など、**書き方が難しい**のが難点

結論

C RTP を置き換えるのは
コンセプトではなく `deducing this`

なぜ「コンセプト」は代わりにならない？

2 つは、**そもそも仕事が違う** 道具だから。

道具	役割	ひとことで
コンセプト C++20	型が条件を満たすか チェック・制約 する	「入口の検査官」
C RTP	基底から派生へ 実装を配る ／静的多態	「実装の配達人」

- コンセプトは「この型は `draw_impl()` を持ってる？」と**確認**はできる
- でも、コードを**代わりに書いて配ってくれるわけではない**

 検査官（コンセプト）は、配達人（C RTP）の仕事まではしてくれない。
だから「コンセプトを覚えた = C RTP不要」とは、単純にはならない。

C RTP の「2つの使い道」と、今風の代わり

C RTP は実は **2つの目的** で使われてきた。目的ごとに“後継”が違う。

C RTP の使い道	いまの置き換え
(A) 共通の実装を配る／静的多態	deducing this C++23
(B) 「この操作を持て」と型を縛る	コンセプト C++20

- 「コンセプトが代わり」と感じたのは **(B) の部分**。そこは正しい！
- でも C RTP の“本丸”は **(A)**。そこを継ぐのが **deducing this**。

次のページで、(A) の before / after を見比べよう。

(A) deducing this で CRTP が“ほどける”

```
// 【before】 CRTP : 型を渡して継承。static_cast が必要
template <class Derived>
struct Shape {
    void draw() const { static_cast<const Derived*>(this)→draw_impl(); }
};
struct Circle : Shape<Circle> { void draw_impl() const; };
```

```
// 【after】 deducing this : ふつうの継承。self が自動で派生型になる
struct Shape {
    void draw(this const auto& self) { self.draw_impl(); } // static_cast 不要!
};
struct Circle : Shape { void draw_impl() const; };           // Shape<Circle> ではない
```

- Shape<Circle> → ただの Shape に、static_cast → 消える
- 第9章で習った「self を推論する」仕組みが、そのまま効いている ✓

(B) 「型を縛る」 だけならコンセプトが素直

C RTP を「共通実装のため」ではなく、単に

“この型は `draw_impl` を持つこと” と縛る 目的で使っていたなら——

```
template <class T>
concept Drawable = requires(const T& t) { t.draw_impl(); };

void render(const Drawable auto& s) { s.draw_impl(); } // 継承ナシで制約できる
```

- 継承関係を作らずに「条件を満たす型だけ受け取る」が書ける
- コンパイルエラーのメッセージも分かりやすい

👉 ここが「コンセプトが C RTP の代わり」と感じた正体。 **半分は当たり！**

それでも CRTP が残る場面もある

deducing this は「**呼ばれるたびに self** を見る」だけ。

だから“**クラスを作る時点で派生の型が要る**”ものは置き換えられない。

```
template <class Derived>           // 派生ごとに別々の値を持たせたい
struct Counter {
    static inline int alive = 0;
    Counter() { ++alive; }
};
struct Enemy : Counter<Enemy> {}; // Enemy 専用のカウンタ
struct Boss  : Counter<Boss>  {}; // Boss 専用のカウンタ (別々に数えられる)
```

- こういう「**型ごとの分身**」を作る用途では、CRTP が今も現役
- 標準ライブラリ（`std::enable_shared_from_this` など）や既存エンジンにも登場する

👉 だから CRTP は「自分で書く」より「**見たら読める**」を目標にすれば十分。

まとめ：どう向き合えばいい？

- ⚠️ 「**コンセプトを習った = C RTP は不要**」ではない（役割が違う道具）
- ✅ C RTP の“本丸” = 静的多態を継ぐのは **deducing this** **C++23**
- ✅ 「型を縛る」用途は **コンセプト** **C++20** が素直
- 📖 残った C RTP は 「**読めればOK**」。無理に自分で書かなくてよい

場面	まず使う道具
新しく静的多態を書く	deducing this
型に「この操作を持て」と要求する	コンセプト
既存コードで Base<Derived> を見かけた	「C RTP だ」と 読めれば十分

まとめの一言

「古い / 新しい」で悩まず、
「いまなら何で書くか」で選べばいい。

C RTP は敵じゃない。読めれば怖くない 🎮